

Data Architecture Reference

A measured reference for data architecture and state design

Edition 1 (draft)

Built from commit repo/book-evidence-registry-v2-6-g9b04419 (9b044195316b62973520dd8e5d620e8f1141b2e4)
— working tree false.n Typst 0.15.1 (unknown commit) (image
sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f).n Evidence registry book/evidence/v2/
claims.json, digest 976e3635f5d2f769c3ab71e35e13083dc13bc728a82be32e10a7bfbe5f12e756.

1 Edition and provenance

This is a generated artefact: Typst source plus the evidence registry, compiled in a pinned Podman image. **Every claim in this book must resolve to a claim in the v2 registry**, and every number must resolve to a measured cell in a cited report. Nothing here is hand-entered at build time; the fields below are injected from the build script.

Source commit	9b044195316b62973520dd8e5d620e8f1141b2e4
Source describe	repo/book-evidence-registry-v2-6-g9b04419
Working tree at build	false
Built at (UTC)	2026-09-22T19:12:08Z
Typst version	0.15.1 (unknown commit)
Typst image digest	sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f
Evidence registry	book/evidence/v2/claims.json
Evidence digest	976e3635f5d2f769c3ab71e35e13083dc13bc728a82be32e10a7bfbe5f12e756
Source tree hash	c35e036e023604b5ee64e081a33afbf23c1dff91d8d58cf8dc5c0982a17348a2

The book is assembled with Typst inside a pinned Podman image; the evidence index at the end is rendered from the registry file at build time.

Coverage gap

Environment and provenance limits travel with every number: all results come from one host (host-zenbook-ux5406sa) with shared cores and no real network between “cluster” nodes. Run tags mark the **producing code state**, not a tree containing the results. A reader reproducing a run must check out the run tag to get the code, then read the committed results directory.

2 How to read a number here

- Every family, variant, topology and scenario section follows the same progressive structure: quick choice, where it thrives and perishes, common scenarios, mechanism, evidence and reproduction.
- A **direct** callout is measured in the cited run. An **analogy** callout is a labelled transfer to a family that was **not** measured. A **gap** callout is missing evidence.
- Gaps are never null results: “not measured” must not be read as “would not change this”.
- No cross-study numeric ranking appears anywhere; no comparability record exists.

Contents

1	Edition and provenance	3
2	How to read a number here	4
3	How to choose	6
3.1	Answerability before performance	6
3.2	The decision path	6
4	Major families	7
4.1	normalized facts	7
4.2	rolldown / duplicated data	8
4.3	rollups / materialized aggregates	8
4.4	embedded documents	9
4.5	append-only history / snapshots	9
4.6	external read copies / caches	10
5	Minor variants	12
6	Cross-family comparisons	13
7	Topologies	14
8	Scenarios	15
9	Evidence and reproduction	16
9.1	The provenance model	16
9.2	Reproducing the book itself	16
9.3	Reproducing a measured run	16
10	Concepts	17
11	Information placement	18
12	Indexes and hot rows	19
13	Concurrency control	20
14	Derived state and history	21
15	Expiry and clock authority	22
16	Cache consistency	23
17	Distributed placement	24
18	Reading benchmarks honestly	25
19	Evidence registry (active v2 claims)	26
19.1	normalized facts	26
19.2	rolldown / duplicated data	27
19.3	rollups / materialized aggregates	27
19.4	embedded documents	28
19.5	append-only history / snapshots	29
19.6	external read copies / caches	30
19.7	Retired v1 claims	30

3 How to choose

CHAPTER · HOW TO CHOOSE

This chapter holds the decision path a reader follows before any family detail. Its prose is written by the HIGH book-synthesis-v1 task; the structure and the evidence hooks below are fixed here.

3.1 Answerability before performance

Direct evidence

The first question is what the schema must be able to **answer**, not how fast it answers. Two measured answerability failures make this a rule rather than a preference.

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

3.2 The decision path

- Name the questions the data must answer, including the ones no current report answers.
- Place each fact once, in the family that answers its questions most directly.
- Choose an arbitration mechanism for the contended surface.
- Accept a maintenance cost for any derived copy, or do not create it.
- State the evidence level and the coverage gap beside the recommendation.

4 Major families

CHAPTER · MAJOR FAMILIES

The six major families are fixed by the goal. Each section follows the same progressive structure; prose is written by book-synthesis-v1, evidence is attached here.

4.1 normalized facts

- 4.1.1 Quick choice in plain language
- 4.1.2 Where it thrives and where it perishes
- 4.1.3 Common scenarios
- 4.1.4 Reusable mechanism
- 4.1.5 Evidence and reproduction

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

4.2 rolldown / duplicated data

4.2.1 Quick choice in plain language

4.2.2 Where it thrives and where it perishes

4.2.3 Common scenarios

4.2.4 Reusable mechanism

4.2.5 Evidence and reproduction

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replications, 5 fresh-load trials per cell, 1 inner iterations.

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05's placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today's standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

4.3 rollups / materialized aggregates

4.3.1 Quick choice in plain language

4.3.2 Where it thrives and where it perishes

4.3.3 Common scenarios

4.3.4 Reusable mechanism

4.3.5 Evidence and reproduction

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing d20_recency_flag with d22_recency_rollup_idx in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-02-study01-d20d22-write-axis

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: studies/01-charity-tree/reports/20260916T090036Z-v3.md · run 20260916T090036Z-v3 · tag run/01-charity-tree/20260916T090036Z-v3

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

4.4 embedded documents

4.4.1 Quick choice in plain language

4.4.2 Where it thrives and where it perishes

4.4.3 Common scenarios

4.4.4 Reusable mechanism

4.4.5 Evidence and reproduction

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

4.5 append-only history / snapshots

4.5.1 Quick choice in plain language

4.5.2 Where it thrives and where it perishes

4.5.3 Common scenarios

4.5.4 Reusable mechanism

4.5.5 Evidence and reproduction

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1-2.3x of the sell-out race on PostgreSQL and 1.8-2.2x on YugabyteDB with tight spreads, 2.1-2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 128 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

4.6 external read copies / caches

4.6.1 Quick choice in plain language

4.6.2 Where it thrives and where it perishes

4.6.3 Common scenarios

4.6.4 Reusable mechanism

4.6.5 Evidence and reproduction

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

5 Minor variants

CHAPTER · MINOR VARIANTS

A minor variant is one controlled change inside a family: index selection, full versus bounded embedding, trigger versus application maintenance, optimistic versus pessimistic concurrency, locks/CAS/isolation, pre-created versus on-demand inventory, cache-aside versus write-through, and strict versus relaxed freshness. Only the first and the concurrency variants have measured controlled pairs in the corpus; the rest are labelled with their gap or analogy status.

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

6 Cross-family comparisons

CHAPTER · CROSS-FAMILY COMPARISONS

Comparisons here are controlled pairs or explicitly labelled analogy. No cross-study numeric ranking appears: no comparability record exists. The six confounds that bound every comparison are published beside the numbers, not in a footnote.

Coverage gap

Several family contrasts are family/package comparisons rather than one-decision pairs; see the confound register book/evidence/v2/confounds.json.

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

7 Topologies

CHAPTER · TOPOLOGIES

Topology is a separate axis from design family: node count, replication, placement, endpoint routing, resource framing, network conditions and failure behaviour. The measured corpus covers PostgreSQL single-node and YugabyteDB single-node/3-node RF=3; placement and real-network coverage are gaps.

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05's placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today's standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-07-real-network-balanced-endpoints

normalized facts · gap

Every multi-node result runs on one laptop with no real network, and balanced client access across cluster endpoints is not proven. Multi-node ratios are placement experiments, not distributed-systems results.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: no real network between nodes; endpoint distribution across the cluster is not measured

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-03-study03-layout-family-package](#)

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

8 Scenarios

CHAPTER · SCENARIOS

Scenarios group the studies by the reader’s domain: a charity donation tree, ticket overbooking and refunds, reserved seating and holds, a configuration portal, and an external cache. Each carries its own answerability questions and its own coverage gaps.

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model’s cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: **conf-04-study05-db-only-cache**

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models’ rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

9 Evidence and reproduction

CHAPTER · EVIDENCE AND REPRODUCTION

Every number in this book resolves to a claim in `book/evidence/v2/claims.json`, and every claim to a measured cell in a cited report. This chapter explains the confidence cards, the run-tag provenance model, and how to reproduce a run in Podman.

9.1 The provenance model

Direct evidence

A run tag marks the producing code state, not a tree that contains the run's results. Check out the tag for the code, then read the committed results directory for the data.

v2-gap-06-native-datastore-families

embedded documents · gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: do not transfer the embedded-document or cache results to a native family

Source: `studies/05-cache-consistency/reports/20260921T-survey3.md` · run `20260921T-survey3` · tag `run/05-cache-consistency/20260921T-survey3`

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: `studies/02-ticket-booking/reports/20260920T234953Z.md` · run `20260920T234953Z` · tag `run/02-ticket-booking/20260920T234953Z`

9.2 Reproducing the book itself

The book is built in Podman from a pinned toolchain; the manifest records every input and the resulting hash.

```
book/build.sh          # builds the pinned image, compiles, verifies, writes the
                        manifest
cat book/dist/build-manifest.json
```

9.3 Reproducing a measured run

```
git checkout run/<study>/<run-id>    # the producing code state
infra/<topology>.sh up                # take the benchmark lock first
cd studies/<study> && ./run-<engine>.sh
```

A run's results live in `studies/<study>/results/<run-id>/`; the report beside them is generated, never hand-edited, and its inputs digest identifies the data.

10 Concepts

11 Information placement

CONCEPT · INFORMATION PLACEMENT

Where a fact lives decides what it can answer and what it costs to change. This concept frames the derive → index → materialize → copy/cache ladder and the answerability-first rule.

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

12 Indexes and hot rows

CONCEPT · INDEXES AND HOT ROWS

An index is the cheapest derived structure, but it is not free: it charges every write and it does not answer a question the schema cannot express. The hot-row case shows the arbitration bound.

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: [studies/02-ticket-booking/reports/20260913T021206Z.md](#) · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

13 Concurrency control

CONCEPT · CONCURRENCY CONTROL

Optimistic (version-checked) and pessimistic (lock-before-update) strategies under the same business invariant. Both local engine mechanisms and application retry loops belong here.

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

14 Derived state and history

CONCEPT · DERIVED STATE AND HISTORY

Rollups, duplicated keys and append-only histories are all derived state with different maintenance costs and different answerability. History is the only one that can answer a question after the event was undone.

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

15 Expiry and clock authority

CONCEPT · EXPIRY AND CLOCK AUTHORITY

Expiry is a write that no client issues; whose clock decides matters. The corpus exposes the difference but cannot answer the funnel that would size a hold window.

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

16 Cache consistency

CONCEPT · CACHE CONSISTENCY

A cache is a read copy with a freshness contract. Invalidation is a fence, not a deletion; publication must be refused when the fence moved. A private in-memory lease does not coordinate instances sharing a store.

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · [run 20260921T-survey3](#) · [tag run/05-cache-consistency/20260921T-survey3](#)

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · [run 20260921T-survey3](#) · [tag run/05-cache-consistency/20260921T-survey3](#)

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · [run 20260921T-survey3](#) · [tag run/05-cache-consistency/20260921T-survey3](#)

17 Distributed placement

CONCEPT · DISTRIBUTED PLACEMENT

Node count, replication and actual data placement are separate facts. The corpus measures node count and replication but does not verify placement or a real network.

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05's placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today's standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-07-real-network-balanced-endpoints

normalized facts · gap

Every multi-node result runs on one laptop with no real network, and balanced client access across cluster endpoints is not proven. Multi-node ratios are placement experiments, not distributed-systems results.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: no real network between nodes; endpoint distribution across the cluster is not measured

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

18 Reading benchmarks honestly

CONCEPT · READING BENCHMARKS

Closed vs open loop, single vs repeated trials, per-node vs equal-total framing, throttled cores, and the difference between a design effect and a cell-order effect. This concept is the reader’s guard against quoting a number the run cannot support.

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: [studies/02-ticket-booking/reports/20260920T234953Z.md](#) · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-04-study05-unmeasured-regimes

external read copies / caches · gap

Real-TTL churn, medium scale, repeated trials, cache resource accounting, YugabyteDB/cluster, verified placement and open-loop demand are unmeasured for Study 05.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: owned by task-20260922T025912Z-study05-v2-protocol

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

19 Evidence registry (active v2 claims)

The active registry holds 23 claims. This index is rendered from the registry file at build time; the digest above identifies its content.

19.1 normalized facts

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: [studies/02-ticket-booking/reports/20260913T021206Z.md](#) · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: [studies/03-reserved-seating/reports/20260915T002411Z.md](#) · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-03-study04-missing-designs

normalized facts · gap

Eight of the eighteen planned Study 04 configuration-portal designs are not implemented, and cardinality, cadence/churn, repeated randomized trials, equal-total resources, placement and YugabyteDB are unmeasured. These are coverage gaps, not null results.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: no conclusion may read 'the missing designs would not change this'; owned by task-20260922T025912Z-study04-v2-protocol

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-07-real-network-balanced-endpoints

normalized facts · gap

Every multi-node result runs on one laptop with no real network, and balanced client access across cluster endpoints is not proven. Multi-node ratios are placement experiments, not distributed-systems results.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: no real network between nodes; endpoint distribution across the cluster is not measured

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.2 rolldown / duplicated data

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replications, 5 fresh-load trials per cell, 1 inner iterations.

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05's placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today's standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.3 rollups / materialized aggregates

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing `d20_recency_flag` with `d22_recency_rollup_idx` in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-02-study01-d20d22-write-axis](#)

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: [studies/01-charity-tree/reports/20260916T090036Z-v3.md](#) · [run 20260916T090036Z-v3](#) · [tag run/01-charity-tree/20260916T090036Z-v3](#)

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (`n3_rollup_trigger`) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (`n4_rollup_app`) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · [run 20260921T1215Z-small](#) · [tag run/04-configuration-portal/20260921T1215Z-small](#)

19.4 embedded documents

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · [run 20260912-small](#) · [tag none](#) (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-03-study03-layout-family-package](#)

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: [studies/03-reserved-seating/reports/20260921T231328Z.md](#) · [run 20260921T231328Z](#) · [tag run/03-reserved-seating/20260921T231328Z](#)

v2-gap-06-native-datastore-families

embedded documents · gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: do not transfer the embedded-document or cache results to a native family

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · [run 20260921T-survey3](#) · [tag run/05-cache-consistency/20260921T-survey3](#)

19.5 append-only history / snapshots

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1-2.3x of the sell-out race on PostgreSQL and 1.8-2.2x on YugabyteDB with tight spreads, 2.1-2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 128 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.6 external read copies / caches

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-gap-04-study05-unmeasured-regimes

external read copies / caches · gap

Real-TTL churn, medium scale, repeated trials, cache resource accounting, YugabyteDB/cluster, verified placement and open-loop demand are unmeasured for Study 05.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: owned by task-20260922T025912Z-study05-v2-protocol

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

19.7 Retired v1 claims

- [claim-gap-04-study02-03-validation-open](#) — The validation it describes as open was completed and integrated 2026-09-22 (task-20260922T025912Z-study02-03-v2-validation). Not carried into v2.